VILORA
API DOCUMENTATION

Production-Standard Backend API Contract

Spring Boot • Java • Spring Security • JWT • RBAC • JPA/Hibernate • PostgreSQL/MySQL

DOCUMENT VERSION	2.0
STATUS	Development Baseline / API Contract
BASE PATH	/api
PRIMARY CLIENT	Vilora React Web Application
DATABASE	PostgreSQL (recommended) / MySQL
AUTH	JWT Access Token + Refresh Token

Prepared as the shared contract for the two-member Vilora backend team. The structure follows the supplied G2_APIS reference: introduction, authentication, pagination, endpoint-by-endpoint request/response definitions, and consistent status-code documentation.

Important: this document is the contract to agree on before implementation. If a new requirement appears during development, update this document first, then implement the change.

DOCUMENT CONTROL & CONVENTIONS

Scope. This document defines the REST API contract for the Vilora dry-fruits / natural-products e-commerce application. It covers customer authentication, catalogue, cart, wishlist, addresses, orders, payments, users and reviews, plus the security and API conventions required to implement them consistently.

Reference format. The supplied G2_APIS.pdf is used as the formatting and documentation reference. Its introduction defines authentication, client identification and pagination, and its endpoint pages document request parameters, request bodies and success responses. ■filecite■turn5file3■

Contract rule. Endpoint names, request fields, response fields, authorization requirements and validation rules should be treated as shared team decisions. Do not change them independently in controller code.

1. API BASE & GENERAL RULES

Base URL: `/api`

Content-Type: `application/json` for JSON requests/responses.

Character encoding: UTF-8.

Resource identifiers: Numeric database IDs may be used initially. UUIDs can be adopted later if the team decides they are required; changing identifier strategy is a contract change.

Dates: ISO-8601 timestamps in UTC, e.g. `2026-09-03T10:30:00Z`.

Money: represent as decimal values in API JSON; never use floating-point arithmetic for final financial calculations. Persist monetary values with database decimal/numeric types.

Soft deletion: products/categories should preferably use an `active` or `deletedAt` strategy instead of physically deleting records that are referenced by orders.

2. AUTHENTICATION

Protected requests use a JWT access token in the Authorization header:

```
Authorization: Bearer <access-token>
```

Access tokens should be short-lived. Refresh tokens are used to obtain a new access token without forcing the user to log in again. Refresh-token storage/rotation strategy must be decided in the security implementation.

3. CLIENT IDENTIFICATION

For the initial web application, the backend may accept an optional client header for tracing or future mobile support:

```
X-Vilora-Client: web
```

If the team later introduces a mobile client, use `mobile`. This is an application-identification header, not a replacement for authentication.

4. PAGINATION, FILTERING & SORTING

List APIs should support pagination where a collection can grow. Following the supplied G2_APIS convention, the default page size is 10 and the maximum is 50. ■filecite■turn5file3■

Parameter	Default	Maximum	Use
page	1	—	1-based page number
perPage	10	50	Items returned per page
sortBy	endpoint default	—	Allowed field whitelist only

Parameter	Default	Maximum	Use
sortOrder	desc/endpoint default	—	asc or desc

Paginated responses use a consistent metadata object. The G2 reference returns `total`, `page` and `perPage` metadata. ■filecite■turn5file2■

```
{
"statusCode": 200,
"data": {
"items": [],
"metadata": {
"total": 0,
"page": 1,
"perPage": 10,
"totalPages": 0
}
}
}
```

Filtering must be implemented server-side. Sort fields must be whitelisted; never concatenate an arbitrary client-supplied field into SQL.

5. STANDARD RESPONSE CONTRACT

Success responses use the G2-style top-level `statusCode` and optional `data`. The exact resource payload is defined per endpoint.

```
{
"statusCode": 200,
"data": {
"id": 1
}
}
```

Recommended error contract:

```
{
"statusCode": 400,
"message": "Validation failed",
"errors": [
{
"field": "email",
"message": "Must be a valid email address"
}
],
"timestamp": "2026-09-03T10:30:00Z",
"path": "/api/auth/register"
}
```

6. STANDARD HTTP STATUS CODES

Code	Meaning	Vilora usage
200	OK	Successful GET, PUT, PATCH or action
201	Created	Successful POST that creates a resource
204	No Content	Optional for successful DELETE where no response body is needed
400	Bad Request	Malformed request / validation failure
401	Unauthorized	Missing/invalid/expired authentication

Code	Meaning	Vilora usage
403	Forbidden	Authenticated but insufficient role/permission
404	Not Found	Resource does not exist
409	Conflict	Duplicate or state conflict
422	Unprocessable Entity	Optional for domain validation failures
500	Internal Server Error	Unexpected server failure

TABLE OF CONTENTS

1. API Base & General Rules

2. Authentication

3. Client Identification

4. Pagination, Filtering & Sorting

5. Standard Response Contract

6. Standard HTTP Status Codes

7. AUTH APIs

8. CATEGORY APIs

9. PRODUCT APIs

10. CART APIs

11. WISHLIST APIs

12. ADDRESS APIs

13. ORDER APIs

14. PAYMENT APIs

15. USER APIs

16. REVIEW APIs

17. Security & RBAC Matrix

18. Validation Rules

19. Database / Entity Relationship Baseline

20. Frontend Integration Contract

21. Production Readiness Checklist

22. Implementation Sequence for Two Backend Developers

7. AUTH APIs

Authentication endpoints are public except `/me`. The G2 reference also documents refresh and logout operations, so Vilora includes them as production-standard additions. ■filecite■turn6file4■ ■filecite■turn6file0■

1. POST /api/auth/register

REGISTER A NEW CUSTOMER

TYPE	POST	AUTHENTICATION	PUBLIC

REQUEST BODY

```
{
"name": "Ananya Mehta",
"email": "ananya@example.com",
"password": "StrongPassword123"
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 201,
"data": {
"userId": 1
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
409	Email already registered	{"statusCode":409,"message":"Email already exists"}

IMPLEMENTATION NOTES

Hash the password with a strong password-hashing algorithm. Never return the password or password hash to the client.

2. POST /api/auth/login

AUTHENTICATE CUSTOMER OR ADMIN

TYPE	POST	AUTHENTICATION	PUBLIC

REQUEST BODY

```
{
"email": "ananya@example.com",
"password": "StrongPassword123"
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"userId": 1,
"accessToken": "jwt-access-token",
"refreshToken": "refresh-token",
"role": "CUSTOMER"
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid credentials	{"statusCode":400,"message":"Invalid email or password"}

3. GET /api/auth/me

RETURN THE AUTHENTICATED USER IDENTITY

TYPE	GET	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"userId": 1,
"name": "Ananya Mehta",
"email": "ananya@example.com",
"role": "CUSTOMER"
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

4. POST /api/auth/refresh

ISSUE A NEW ACCESS TOKEN

TYPE	POST	AUTHENTICATION	PUBLIC + VALID REFRESH TOKEN

REQUEST BODY

```
{
"refreshToken": "refresh-token"
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"accessToken": "new-jwt-access-token",
"refreshToken": "new-refresh-token"
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
401	Invalid/expired/revoked refresh token	{"statusCode":401,"message":"Invalid refresh token"}

IMPLEMENTATION NOTES

Prefer refresh-token rotation and server-side revocation tracking for a production deployment.

5. POST /api/auth/logout

LOGOUT / REVOKE REFRESH-TOKEN SESSION

TYPE	POST	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"refreshToken": "refresh-token"
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
```

```
"message": "Logged out successfully"
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

8. CATEGORY APIs

6. GET /api/categories

RETRIEVE ALL ACTIVE CATEGORIES

TYPE	GET	AUTHENTICATION	PUBLIC

REQUEST QUERY PARAMS

Parameter	Type	Required	Description
page	int	optional	Page number; default 1
perPage	int	optional	Items per page; default 10, maximum 50
search	string	optional	Search category name
sortBy	string	optional	name or createdAt
sortOrder	string	optional	asc or desc

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"categories": [
{"id": 1, "name": "Dry Fruits", "description": "Premium dry fruits"}
],
"metadata": {"total": 6, "page": 1, "perPage": 10, "totalPages": 1}
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}

IMPLEMENTATION NOTES

Public listing returns active categories only. Admin-created/updated categories require ADMIN role.

7. GET /api/categories/{id}

RETRIEVE ONE CATEGORY

TYPE	GET	AUTHENTICATION	PUBLIC

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"id": 1,
"name": "Dry Fruits",
"description": "Premium dry fruits",
"productCount": 4
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}

HTTP	Condition	Example
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}

8. POST /api/categories

CREATE A CATEGORY

TYPE	POST	AUTHENTICATION	JWT + ADMIN

REQUEST BODY

```
{
"name": "Dry Fruits",
"description": "Premium dry fruits"
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 201,
"data": {"categoryId": 1}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}

IMPLEMENTATION NOTES

Category name should be unique (case-insensitive) according to the database/business rule.

9. PUT /api/categories/{id}

REPLACE/UPDATE CATEGORY FIELDS

TYPE	PUT	AUTHENTICATION	JWT + ADMIN

REQUEST BODY

```
{
"name": "Dry Fruits",
"description": "Premium organic dry fruits"
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"categoryId": 1}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

HTTP	Condition	Example
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}

10. DELETE /api/categories/{id}

DEACTIVATE/DELETE CATEGORY

TYPE	DELETE	AUTHENTICATION	JWT + ADMIN

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"categoryId": 1}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}

IMPLEMENTATION NOTES

Prefer a soft-delete/deactivation approach when products reference the category.

9. PRODUCT APIs

11. GET /api/products

RETRIEVE PRODUCTS WITH CATALOGUE CONTROLS

TYPE	GET	AUTHENTICATION	PUBLIC

REQUEST QUERY PARAMS

Parameter	Type	Required	Description
page	int	optional	Default 1
perPage	int	optional	Default 12 for the Shop UI; maximum 50
search	string	optional	Search product name/description
categoryId	long	optional	Filter by category
minPrice	decimal	optional	Minimum selling price
maxPrice	decimal	optional	Maximum selling price
rating	decimal	optional	Minimum rating, e.g. 4
inStock	boolean	optional	Only products with stock > 0
sortBy	string	optional	relevance, price, rating, createdAt, name
sortOrder	string	optional	asc or desc

ON SUCCESS RESPONSE


```
{
"statusCode": 200,
"data": {
"products": [
{
"id": 1,
"name": "Himalayan Almonds",
"description": "Crunchy, naturally sweet...",
"price": 649.00,
"originalPrice": 799.00,
"discountPercent": 19,
"stockQuantity": 25,
"imageUrl": "/images/almonds.jpg",
"categoryId": 1,
"rating": 4.5,
"reviewCount": 124,
"active": true
}
],
"metadata": {"total": 18, "page": 1, "perPage": 12, "totalPages": 2}
}
}
```


COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}

IMPLEMENTATION NOTES

This endpoint directly supports the React Shop page shown by the team: category filters, price range, ratings, availability, search, sorting and pagination. Use Spring Data Pageable + Specification/Criteria/Querydsl-style predicates as the implementation grows.

12. GET /api/products/{id}

RETRIEVE PRODUCT DETAILS

TYPE	GET	AUTHENTICATION	PUBLIC

ON SUCCESS RESPONSE


```
{
"statusCode": 200,
"data": {
"id": 1,
"name": "Himalayan Almonds",
"description": "Crunchy, naturally sweet and rich in nutrients.",
"price": 649.00,
"originalPrice": 799.00,
"discountPercent": 19,
"stockQuantity": 25,
"imageUrl": "/images/almonds.jpg",
"category": {"id": 1, "name": "Dry Fruits"},
"rating": 4.5,
"reviewCount": 124,
"active": true
}
}
```


COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}

13. POST /api/products

CREATE A PRODUCT

TYPE	POST	AUTHENTICATION	JWT + ADMIN

REQUEST BODY


```
{
"name": "Himalayan Almonds",
"description": "Crunchy, naturally sweet and rich in nutrients.",
"price": 649.00,
"originalPrice": 799.00,
"stockQuantity": 25,
"imageUrl": "/images/almonds.jpg",
"categoryId": 1
}
```


ON SUCCESS RESPONSE


```
{
"statusCode": 201,
"data": {"productId": 1}
}
```


COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

HTTP	Condition	Example
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}

IMPLEMENTATION NOTES

Validate price >= 0, originalPrice >= price when supplied, stockQuantity >= 0, valid category and non-blank name. Never accept rating/reviewCount from the client.

14. PUT /api/products/{id}

UPDATE A PRODUCT

TYPE	PUT	AUTHENTICATION	JWT + ADMIN

REQUEST BODY

```
{
"name": "Himalayan Almonds",
"description": "Updated description",
"price": 649.00,
"originalPrice": 799.00,
"stockQuantity": 30,
"imageUrl": "/images/almonds.jpg",
"categoryId": 1,
"active": true
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"productId": 1}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}

15. DELETE /api/products/{id}

DEACTIVATE A PRODUCT

TYPE	DELETE	AUTHENTICATION	JWT + ADMIN

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"productId": 1}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}

HTTP	Condition	Example
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}

IMPLEMENTATION NOTES

Prefer deactivation/soft deletion so historical order items retain their product reference and pricing history.

10. CART APIs

16. GET /api/cart

RETRIEVE CURRENT USER'S CART

TYPE	GET	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"cartId": 1,
"items": [
{
"id": 10,
"productId": 1,
"productName": "Himalayan Almonds",
"imageUrl": "/images/almonds.jpg",
"unitPrice": 649.00,
"quantity": 1,
"lineTotal": 649.00
}
],
"subtotal": 649.00,
"itemCount": 1
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

IMPLEMENTATION NOTES

Recalculate line totals and subtotal from current backend product data. Do not trust totals supplied by the frontend.

17. POST /api/cart/items

ADD A PRODUCT TO THE CART

TYPE	POST	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"productId": 1,
"quantity": 2
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 201,
"data": {"cartItemId": 10}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}

HTTP	Condition	Example
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}

IMPLEMENTATION NOTES

Verify product exists, is active, is in stock, and has enough inventory. If the product already exists in the cart, define the contract to increase quantity rather than create a duplicate line.

18. PATCH /api/cart/items/{id}

CHANGE CART ITEM QUANTITY

TYPE	PATCH	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"quantity": 3
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"cartItemId": 10, "quantity": 3}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}

IMPLEMENTATION NOTES

Quantity must be a positive integer and must not exceed available stock.

19. DELETE /api/cart/items/{id}

REMOVE ONE CART ITEM

TYPE	DELETE	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"cartItemId": 10}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

HTTP	Condition	Example
404	Resource not found	{"statusCode":404,"message":"Resource not found"}

20. DELETE /api/cart

CLEAR CURRENT USER'S CART

TYPE	DELETE	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"message": "Cart cleared"}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

11. WISHLIST APIs

21. GET /api/wishlist

RETRIEVE CURRENT USER'S WISHLIST

TYPE	GET	AUTHENTICATION	JWT REQUIRED

REQUEST QUERY PARAMS

Parameter	Type	Required	Description
page	int	optional	Default 1
perPage	int	optional	Default 10, maximum 50

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"items": [],
"metadata": {"total": 0, "page": 1, "perPage": 10, "totalPages": 0}
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

22. POST /api/wishlist/{productId}

ADD A PRODUCT TO WISHLIST

TYPE	POST	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 201,
"data": {"productId": 1}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}

23. DELETE /api/wishlist/{productId}

REMOVE A PRODUCT FROM WISHLIST

TYPE	DELETE	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
```

```
"data": {"productId": 1}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}

12. ADDRESS APIs

24. GET /api/addresses

RETRIEVE CURRENT USER'S SAVED ADDRESSES

TYPE	GET	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"addresses": [
{
"id": 1,
"fullName": "Ananya Mehta",
"addressLine1": "24 Green Park",
"addressLine2": "Andheri West",
"city": "Mumbai",
"state": "Maharashtra",
"pincode": "400053",
"phone": "+919876543210",
"isDefault": true
}
]
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

25. GET /api/addresses/{id}

RETRIEVE ONE SAVED ADDRESS

TYPE	GET	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"id": 1,
"fullName": "Ananya Mehta",
"addressLine1": "24 Green Park",
"city": "Mumbai",
"state": "Maharashtra",
"pincode": "400053",
"phone": "+919876543210",
"isDefault": true
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

HTTP	Condition	Example
404	Resource not found	{"statusCode":404,"message":"Resource not found"}

26. POST /api/addresses

CREATE A SAVED ADDRESS

TYPE	POST	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"fullName": "Ananya Mehta",
"addressLine1": "24 Green Park",
"addressLine2": "Andheri West",
"city": "Mumbai",
"state": "Maharashtra",
"pincode": "400053",
"phone": "+919876543210",
"isDefault": true
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 201,
"data": {"addressId": 1}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

IMPLEMENTATION NOTES

Validate Indian pincode/phone according to the final business rule. If isDefault=true, unset the previous default address in one transaction.

27. PUT /api/addresses/{id}

UPDATE SAVED ADDRESS

TYPE	PUT	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"fullName": "Ananya Mehta",
"addressLine1": "24 Green Park",
"addressLine2": "Andheri West",
"city": "Mumbai",
"state": "Maharashtra",
"pincode": "400053",
"phone": "+919876543210",
"isDefault": true
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"addressId": 1}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}

28. DELETE /api/addresses/{id}

DELETE A SAVED ADDRESS

TYPE	DELETE	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"addressId": 1}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}

IMPLEMENTATION NOTES

Do not allow deletion of the only address when the application has an active checkout requirement unless checkout permits a new address.

13. ORDER APIs

29. POST /api/orders

CREATE AN ORDER FROM THE CURRENT CART

TYPE	POST	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"addressId": 1,
"promoCode": "WELCOME10",
"paymentMethod": "ONLINE"
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 201,
"data": {
"orderId": 1001,
"paymentId": 501,
"subtotal": 649.00,
"discount": 0.00,
"tax": 32.45,
"shipping": 0.00,
"total": 681.45,
"status": "PENDING_PAYMENT"
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}
422	Insufficient stock / invalid order state	{"statusCode":422,"message":"Insufficient stock"}

IMPLEMENTATION NOTES

Critical: never trust product prices, tax, shipping, discount or total from the frontend. Load product prices from the database, validate stock, calculate totals server-side and use a database transaction. Consider inventory locking/atomic stock updates to prevent overselling.

30. GET /api/orders

RETRIEVE CURRENT USER'S ORDERS

TYPE	GET	AUTHENTICATION	JWT REQUIRED

REQUEST QUERY PARAMS

Parameter	Type	Required	Description
page	int	optional	Default 1
perPage	int	optional	Default 10, maximum 50
status	string	optional	PENDING_PAYMENT, PAID, PROCESSING, SHIPPED, DELIVERED, CANCELLED

Parameter	Type	Required	Description
sortBy	string	optional	createdAt or total
sortOrder	string	optional	asc or desc

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"orders": [
{
"id": 1001,
"status": "PAID",
"total": 681.45,
"createdAt": "2026-09-03T10:30:00Z"
}
],
"metadata": {"total": 1, "page": 1, "perPage": 10, "totalPages": 1}
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

31. GET /api/orders/{id}

RETRIEVE ORDER DETAILS

TYPE	GET	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"id": 1001,
"status": "PAID",
"items": [
{
"productId": 1,
"productName": "Himalayan Almonds",
"quantity": 1,
"unitPrice": 649.00,
"lineTotal": 649.00
}
],
"subtotal": 649.00,
"discount": 0.00,
"tax": 32.45,
"shipping": 0.00,
"total": 681.45,
"shippingAddress": {},
"createdAt": "2026-09-03T10:30:00Z"
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}

HTTP	Condition	Example
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}

IMPLEMENTATION NOTES

Authorization must verify that the order belongs to the authenticated user, unless the caller has an ADMIN role.

32. PATCH /api/orders/{id}/cancel

CANCEL AN ELIGIBLE ORDER

TYPE	PATCH	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"reason": "Changed my mind"
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"orderId": 1001,
"status": "CANCELLED"
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}
422	Order cannot be cancelled in current status	{"statusCode":422,"message":"Order cannot be cancelled"}

IMPLEMENTATION NOTES

Define the allowed state transitions explicitly. Example: PENDING_PAYMENT/PAID/PROCESSING may have different cancellation rules. Refund handling belongs to the payment module.

14. PAYMENT APIs

The payment provider is intentionally abstract in this API contract. The team should choose a provider such as Razorpay/Stripe only after confirming business and deployment requirements. Payment signatures/webhooks must be verified server-side.

33. POST /api/payments/create

CREATE A PAYMENT ORDER/INTENT FOR AN EXISTING VILORA ORDER

TYPE	POST	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"orderId": 1001
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 201,
"data": {
"paymentId": 501,
"orderId": 1001,
"providerOrderId": "provider-order-id",
"amount": 681.45,
"currency": "INR",
"status": "CREATED"
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}
422	Order is not payable	{"statusCode":422,"message":"Invalid payment state"}

IMPLEMENTATION NOTES

Amount must come from the server-side order total. Never accept a client-supplied payment amount as authoritative.

34. POST /api/payments/verify

VERIFY PROVIDER PAYMENT RESULT/SIGNATURE

TYPE	POST	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"paymentId": 501,
"providerPaymentId": "provider-payment-id",
"signature": "provider-signature"
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"paymentId": 501,
```

```
"status": "PAID",
"orderId": 1001
}
}
```


COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}
422	Signature/amount/order mismatch	{"statusCode":422,"message":"Payment verification failed"}

IMPLEMENTATION NOTES

Verify the provider signature using the provider SDK/secret on the backend. Make the operation idempotent so repeated verification does not duplicate payment state changes.

35. GET /api/payments/{id}

RETRIEVE PAYMENT STATUS

TYPE	GET	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE


```
{
"statusCode": 200,
"data": {
"paymentId": 501,
"orderId": 1001,
"amount": 681.45,
"currency": "INR",
"status": "PAID",
"createdAt": "2026-09-03T10:30:00Z"
}
}
```


COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}

15. USER APIs

36. GET /api/users/me

RETRIEVE CURRENT USER'S PROFILE

TYPE	GET	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"id": 1,
"name": "Ananya Mehta",
"email": "ananya@example.com",
"phone": "+919876543210",
"role": "CUSTOMER"
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

37. PUT /api/users/me

UPDATE CURRENT USER'S PROFILE

TYPE	PUT	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"name": "Ananya Mehta",
"phone": "+919876543210"
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {
"userId": 1
}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}

IMPLEMENTATION NOTES

Do not allow the customer profile endpoint to change role, password hash, account status or other security-sensitive fields. Password change should be a separate authenticated flow if required.

16. REVIEW APIs

38. GET /api/products/{productId}/reviews

RETRIEVE PRODUCT REVIEWS

TYPE	GET	AUTHENTICATION	PUBLIC

REQUEST QUERY PARAMS

Parameter	Type	Required	Description
page	int	optional	Default 1
perPage	int	optional	Default 10, maximum 50
rating	int	optional	Filter by rating 1-5
sortBy	string	optional	createdAt or rating
sortOrder	string	optional	asc or desc

ON SUCCESS RESPONSE


```
{
"statusCode": 200,
"data": {
"reviews": [
{
"id": 50,
"productId": 1,
"userName": "Ananya",
"rating": 5,
"comment": "Excellent quality.",
"createdAt": "2026-09-03T10:30:00Z"
}
],
"metadata": {"total": 124, "page": 1, "perPage": 10, "totalPages": 13}
}
}
```


COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}

39. POST /api/products/{productId}/reviews

CREATE A PRODUCT REVIEW

TYPE	POST	AUTHENTICATION	JWT REQUIRED

REQUEST BODY


```
{
"rating": 5,
"comment": "Excellent quality."
}
```


ON SUCCESS RESPONSE


```
{
"statusCode": 201,
"data": {"reviewId": 50}
}
```


COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}
422	User is not eligible to review	{"statusCode":422,"message":"Purchase required"}

IMPLEMENTATION NOTES

Recommended production rule: only a verified purchaser may review the product, and one review per user per order/product unless editing is explicitly supported.

40. PUT /api/reviews/{id}

UPDATE OWN REVIEW

TYPE	PUT	AUTHENTICATION	JWT REQUIRED

REQUEST BODY

```
{
"rating": 4,
"comment": "Updated comment."
}
```

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"reviewId": 50}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}
409	Duplicate/state conflict	{"statusCode":409,"message":"Conflict"}
403	Review belongs to another user	{"statusCode":403,"message":"Forbidden"}

41. DELETE /api/reviews/{id}

DELETE OWN REVIEW

TYPE	DELETE	AUTHENTICATION	JWT REQUIRED

ON SUCCESS RESPONSE

```
{
"statusCode": 200,
"data": {"reviewId": 50}
}
```

COMMON ERROR CONDITIONS

HTTP	Condition	Example
400	Invalid request / validation failure	{"statusCode":400,"message":"Validation failed"}
401	Missing or invalid JWT	{"statusCode":401,"message":"Unauthorized"}
403	Insufficient role	{"statusCode":403,"message":"Forbidden"}
404	Resource not found	{"statusCode":404,"message":"Resource not found"}

17. SECURITY & RBAC MATRIX

Roles: **CUSTOMER** and **ADMIN**. A user is authenticated by JWT; authorization is then evaluated using the user's role and resource ownership.

Module	PUBLIC	CUSTOMER	ADMIN	Ownership check	Notes
Auth register/login	✓	—	—	—	Public entry points
Categories GET	✓	✓	✓	—	Active catalogue
Categories write	—	—	✓	—	Admin only
Products GET	✓	✓	✓	—	Catalogue
Products write	—	—	✓	—	Admin only
Cart / Wishlist	—	✓	✓	User-owned	Never expose another user's data
Addresses	—	✓	✓	User-owned	Admin access only if explicitly needed
Orders	—	✓	✓	User-owned	Admin may manage operationally
Payments	—	✓	✓	Order-owned	Verify provider signatures
Reviews	GET	✓	✓	User-owned for write	Verified-purchase rule recommended

JWT SECURITY RULES

• Passwords are hashed; plaintext passwords are never logged or returned.
• Access tokens contain only the minimum claims needed (for example subject/user ID and role).
• Refresh tokens must be validated and preferably rotated/revoked server-side.
• CORS should allow only the configured frontend origin(s) in deployed environments.
• Disable verbose stack traces and sensitive error details in production.
• Apply rate limiting/throttling at least to login, registration and payment-related endpoints.
• Secrets belong in environment variables/secret management, never in Git.

18. VALIDATION RULES

Field	Rules	Applies to
name	Required where specified; trim; length limit; no blank value	User, Product, Category
email	Required; valid email format; normalized; unique for account	Register/Login
password	Minimum length/complexity defined by security policy; never return	Register/Login
price	Decimal >= 0; use database NUMERIC/DECIMAL	Product, Order calculation
originalPrice	>= price when discount is represented; >= 0	Product
stockQuantity	Integer >= 0	Product/Inventory
quantity	Integer >= 1 and <= available stock	Cart/Order
rating	Integer 1 through 5	Reviews
pincode	Validate against final Indian address policy	Address
sortBy	Must match an allow-list; reject unknown fields	List APIs
page/perPage	page >= 1; perPage >= 1; perPage <= 50	Paginated APIs

19. DATABASE / ENTITY RELATIONSHIP BASELINE

Recommended baseline entities for Spring Data JPA/Hibernate:

```
User
1 ■■■■■■■■■ * Address

User
1 ■■■■■■■■■ 1 Cart
|
1
|
*
CartItem * ■■■■■■■■■ 1 Product
|
*
|
1
Category

User
1 ■■■■■■■■■ * WishlistItem * ■■■■■■■■■ 1 Product

User
1 ■■■■■■■■■ * Order
|
1
|
*
OrderItem * ■■■■■■■■■ 1 Product

Order
1 ■■■■■■■■■ * Payment

User
1 ■■■■■■■■■ * Review * ■■■■■■■■■ 1 Product
```

Important order-data rule: OrderItem should store a snapshot of the purchased product name and unit price. Product prices may change later; historical orders must not change.

Recommended database constraints: unique user email; unique category name where appropriate; unique (user_id, product_id) for wishlist; appropriate unique/index constraints for cart items and review eligibility.

Indexes: add indexes for product category, active flag, price, createdAt, order user/status/createdAt, review product/createdAt, and other fields used heavily by filters.

20. FRONTEND INTEGRATION CONTRACT

The supplied React frontend already demonstrates the customer journey: Shop → product filtering → Add to Cart → Cart summary → Checkout → delivery address → payment → confirmation. The backend should expose data and actions that map directly to those screens.

Frontend screen/action	Backend API(s)	Backend responsibility
Shop product grid	GET /api/products	Pagination, search, category/price/rating/stock filters, sorting
Product card	GET /api/products/{id}	Price, discount, stock, rating and product data
Add to Cart	POST /api/cart/items	Validate product/stock and create/update cart item
Cart page	GET /api/cart	Calculate current line totals/subtotal
Quantity +/-	PATCH /api/cart/items/{id}	Validate quantity and stock
Remove item	DELETE /api/cart/items/{id}	Remove only user's item
Wishlist	GET/POST/DELETE /api/wishlist...	Persist wishlist per authenticated user
Saved addresses	GET/POST/PUT/DELETE /api/addresses...	Persist and validate addresses
Proceed to checkout	POST /api/orders	Revalidate cart, stock, address and calculate final amount
Payment	POST /api/payments/create + verify	Create and verify provider payment
Order history	GET /api/orders	Paginated current-user orders
Reviews	GET/POST/PUT/DELETE review APIs	Eligibility, ownership and rating validation

Example frontend request flow

```
1. GET /api/products?page=1&perPage=12&categoryId=1&inStock=true
2. POST /api/cart/items { productId, quantity }
3. GET /api/cart
4. GET /api/addresses
5. POST /api/orders { addressId, promoCode, paymentMethod }
6. POST /api/payments/create { orderId }
7. POST /api/payments/verify { paymentId, providerPaymentId, signature }
8. GET /api/orders/{id}
```

21. PRODUCTION READINESS CHECKLIST

Area	Production requirement
Architecture	Controller → DTO → Service → Repository; business logic stays in services/domain layer.
DTOs	Do not expose JPA entities directly from controllers.
Validation	Use Jakarta Bean Validation plus domain-level validation.
Exception handling	Use @RestControllerAdvice with one consistent error contract.
Security	Spring Security + JWT + role-based authorization + ownership checks.
Database	PostgreSQL recommended; use migrations such as Flyway/Liquibase before production.
Transactions	Use @Transactional around order creation, stock updates, default-address changes and other multi-step writes.
Concurrency	Protect inventory from overselling with appropriate locking/atomic update strategy.
Pagination	Use Pageable/server-side pagination; never load an unbounded product/order/review collection.
Sorting	Whitelist sort fields.
Performance	Index filter/sort columns; avoid N+1 queries; use fetch strategies deliberately.
Logging	Structured logs; never log passwords, JWTs, payment secrets or sensitive personal data.
API docs	Swagger/OpenAPI kept synchronized with the contract.
Configuration	Use profiles/environment variables; no secrets in source control.
CORS	Allow only known frontend origins in deployed environments.
Rate limiting	Protect authentication/payment endpoints.
Payment	Verify signatures/webhooks; make payment operations idempotent.
Observability	Actuator/health checks, metrics and centralized logs for deployed environments.
Testing	Unit + repository + controller/integration tests; test security and order calculations.
Deployment	AWS deployment with HTTPS, managed database, secrets management and backups.

RECOMMENDED SUPPORTING TECHNOLOGY

Spring Boot • Spring Web • Spring Security • Spring Data JPA • Hibernate • Jakarta Validation • PostgreSQL • Maven/Gradle • Lombok • Swagger/OpenAPI • JWT library • Flyway/Liquibase • Spring Boot Actuator • JUnit/Mockito • AWS deployment services.

Spring AI: keep this as an optional module rather than coupling it to core commerce operations. It can later support product search/recommendations, an AI shopping assistant, semantic search or internal catalogue tooling without making checkout dependent on an AI service.

22. IMPLEMENTATION SEQUENCE FOR TWO BACKEND DEVELOPERS

The team should implement in dependency order. Do not start by creating every controller at once.

Phase	Developer A	Developer B	Exit condition
0. Contract	Review API/security rules	Review API/database rules	Both approve v2 contract
1. Project foundation	Spring Security/JWT config	DB config/entities base	Application starts; DB connects
2. Auth + User	Auth controller/service/security	User entity/repository/DTOs	Register/login/me tested
3. Category + Product	Category APIs	Product APIs + filtering	Shop API works in Postman
4. Cart + Wishlist	Cart	Wishlist	Authenticated cart flow works
5. Address	Address APIs	Validation/tests	Checkout can load addresses
6. Order	Order service/controller	Order item + inventory logic/tests	Order total is server-calculated
7. Payment	Payment integration	Payment verification/idempotency/tests	Test payment flow works
8. Reviews	Review APIs	Eligibility/aggregation/tests	Review flow works
9. Integration	React API integration	Bug fixing + integration tests	Frontend journey works end-to-end
10. Production	AWS/security/config	Swagger/testing/observability	Deployable release candidate

Git/team rule: each endpoint change that affects request/response shape must update this document (and Swagger) before or with the code change. Use feature branches and small, focused commits. Avoid editing the same module simultaneously without coordination.

Suggested branch examples: `feature/auth`, `feature/products`, `feature/cart`, `feature/orders`, `feature/payments`.

FINAL API INVENTORY

Module	Endpoints
AUTH	5
CATEGORIES	5
PRODUCTS	5
CART	5
WISHLIST	3
ADDRESSES	5
ORDERS	4
PAYMENTS	3
USERS	2
REVIEWS	4
TOTAL	41

The original team checklist contained 39 endpoints. This v2 contract adds two production-standard authentication operations—refresh and logout—bringing the baseline to **41 endpoints**. The remaining production requirements are handled as conventions, validation, security, transaction, testing and deployment rules rather than by blindly adding more URLs.

This document is intentionally a **baseline**. Coupons/promos, inventory history, admin dashboards, delivery tracking, payment webhooks, notifications and advanced search can be added later as the product requirements become concrete. When they are added, document them here before implementation.

END OF VILORA API DOCUMENTATION v2.0